Designing Zorn Mesh: A Local-First Agent Communication Layer

Executive summary

Zorn Mesh should be positioned as a **local agent message bus and control plane**: a single-machine evented fabric that provides registration, discovery, capabilities, and secure message routing between heterogeneous coding agents and runtimes (Copilot, Claude, Gemini, local agents, etc.). It should feel more like a **local Dapr-style application runtime plus NATS-like core** than a traditional network service mesh or heavyweight orchestrator.[^1][^2][^3][^4][^5]

The recommended architecture is:

- A single **local daemon** (zorn-meshd) exposing a **Unix domain socket (UDS) JSON-RPC 2.0 API** as the primary transport, with optional localhost TCP/WebSocket as an extension.[^6][^7][^8]
- A **canonical message envelope** and **agent capability model** inspired by MCP, A2A, and ACP but tailored for local, high-trust environments and multi-pattern messaging (req/rep, pub/sub, streaming).[^9][^10][^1]
- A **hybrid message bus / event bus** core with simple routing semantics, explicit correlation IDs, and pluggable persistence backed by **SQLite (append-only log tables) for history and traceability**.[^11][^12][^13]
- **At-least-once delivery** with idempotency keys, bounded retries, dead-letter queues, and replay for inspection rather than strict exactly-once guarantees, which are overkill for a single-node local-first system.[^14][^5]
- A **practical local security model** that leans on OS process identity, Unix socket permissions, optional per-agent keys/tokens, and signed envelopes for high-sensitivity use cases, borrowing ideas from NATS system-level proposals and zero-trust agent protocols.[^15][^16][^17]
- A core implementation in **Rust**, with first-class **TypeScript and Python SDKs**, and an optional web dashboard (Next.js/React) talking to the daemon via WebSocket.[^18][^13]

This report walks through each requested dimension and ends with a concrete MVP blueprint, example schemas, topology, decision matrix, and roadmap.

***

1. Problem framing

Category and scope

Zorn Mesh sits primarily in the category of a **local agent message bus and control plane**: it coordinates and routes messages between agents, maintains an agent registry and capabilities, and provides observability and history. It borrows concepts from service meshes (sidecar-like

daemon, mTLS-like channel security) but operates strictly within the local host and focuses on application-level messages, not L4 networking.[^2][^19][^20][^1][^9]

Best description for the core:

- **Agent bus / orchestration substrate (local-first)**
  - Bus: because agents publish/subscribe, send requests, and receive responses via named subjects/channels.
  - Orchestration substrate: because Zorn Mesh manages registration, discovery, and simple routing policy without being a full workflow engine.

It is **not**:

- A full-blown **service mesh** like Istio/Linkerd (no L4 proxying of arbitrary TCP traffic, no transparent sidecars).[^19][^20]
- A general-purpose container orchestrator (no scheduling, resource isolation, or remote nodes).
- A full **orchestrator of agent plans** (it provides the fabric, not the planning logic).

Architecture patterns

Relevant patterns for Zorn Mesh include:

- **Sidecar / local daemon pattern**: a single `zorn-meshd` per user machine, with agents connecting via UDS/TCP.[^4][^2]
- **Message bus + event bus hybrid**: inspired by NATS and Redis pub/sub, with subject-based routing and request/reply semantics.[^5][^17]
- **Central registry + capability catalog** similar to MCP servers and A2A AgentCards.[^21][^1][^9]
- **Log-centric design** for history, akin to local-first systems that record every mutation in SQLite.[^12][^11]

MVP scope vs. deferred features

In scope for MVP:

- Single-host deployment (`zorn-meshd`) with:
  - UDS JSON-RPC 2.0 API for control plane and data plane.
  - Optional localhost HTTP/WebSocket gateway.
- Agent registration, heartbeat, discovery, and capability advertisement.
- Messaging primitives:
  - Direct messaging (agent-to-agent via bus).
  - Request/response (correlation IDs, timeouts).
  - Pub/sub topics.
  - Streaming (chunked responses/events over a logical stream).

- Persistence:
  - Agent registry and capability catalog in SQLite.
  - Append-only message log with retention controls.
  - Basic replay for observability/debugging.
- Reliability:
  - At-least-once semantics with idempotency keys.
  - Retries with backoff, dead-letter queues for failed deliveries.
  - Basic backpressure (bounded queues per agent, flow control).
- Security:
  - UDS permissions + per-agent tokens or shared secrets.
  - Process identity awareness (UID/GID, PID) for policy.[^15]
- Developer experience:
  - CLI (`zorn`) to start/stop daemon, inspect state, tail messages.
  - TypeScript and Python SDKs.
  - Minimal web UI for live traffic and agent list.

Deferred for post-MVP:

- Multi-host topologies (federation or remote peers).
- Strong zero-trust identity (hardware-backed keys, TPM integration).[^16][^15]
- Advanced policy language (OPA-like policies for message routing).
- CRDT-based shared state between agents.
- Built-in workflow engine or planner.
- Pluggable remote transports (NATS, Redis, Kafka) and cloud backends.

***

2. Communication model

Transport comparison for local-only multi-agent systems

Key local transports and fabrics:

- **Direct IPC (shared memory, pipes)**: extremely low latency but not ergonomic for multi-language agent ecosystems and harder to observe and secure consistently.[^8]
- **JSON-RPC over STDIO**: used by LSP and many tools; great for one host/one tool pairs but less ideal for multi-agent mesh (no central registry, hard to multiplex many peers).[^1]
- **Unix domain sockets (UDS)**: lower latency than TCP loopback, leverage filesystem permissions, and expose client UID/PID for security.[^7][^6][^8]
- **localhost HTTP/1.1 / HTTP/2**: familiar, debuggable with curl/browser, wide language support but higher overhead than UDS and more complex framing.[^6][^8]
- **WebSocket over localhost**: good for streaming and browser dashboards, but adds HTTP handshake and framing overhead.[^5]

- **gRPC over UDS/TCP**: efficient, strongly typed, bidirectional streaming; heavier framework, requires generated stubs and TLS plumbing if used beyond local.[^16]
- **NATS**: high-throughput broker with request/reply, pub/sub, queue groups; excellent for distributed systems, but running a full NATS for one machine is heavier and adds operational complexity.[^17][^5]
- **Redis pub/sub**: fast but not designed as a robust message queue for guaranteed delivery; pub/sub is lossy and ephemeral.[^14][^5]
- **SQLite-backed queues / embedded event bus**: local-first persistence and history with SQL visibility; requires building the messaging semantics on top.[^11][^12]

Recommended primary and extension transports

Primary transport: Unix domain socket + JSON-RPC 2.0 with streaming extensions.

- UDS gives low latency, no TCP overhead, no port conflicts, and native PID/UID propagation for auth decisions.[^8][^6][^15]
- JSON-RPC 2.0 is already used by MCP and A2A as a core binding, making it familiar for agent tool builders and easy to integrate.[^9][^21][^1]
- Streaming can be layered on JSON-RPC via logical stream IDs and chunked notifications (similar to how MCP and A2A handle streaming messages).[^1][^9]

Fallback/extension transport: localhost HTTP/WebSocket gateway.

- Expose a thin HTTP/JSON and WebSocket endpoint so agents that cannot speak UDS can still participate.
- Gateway translates HTTP/WebSocket requests into internal JSON-RPC over UDS.
- Keeps core simple (one native protocol) while enabling browser dashboards and non-UDS runtimes.

Tradeoffs

- **Reliability**: UDS + JSON-RPC is as reliable as the host OS; no network partitions, only process crashes to handle.
  - NATS or Redis add broker-level persistence and clustering, but that is unnecessary for local-first MVP and complicates the stack.[^5]
- **Latency**: UDS is faster than TCP loopback and HTTP, making it ideal for high-frequency coding agent interactions.[^22][^6][^8]
- **Debuggability**: JSON messages are easy to log and inspect; HTTP gateway enables curl and browser-based debugging.
- **Security**: UDS permissions, plus process identity inspection in the daemon, provide better control than open TCP ports on localhost.[^15][^8]
- **Language interoperability**: JSON-RPC over stdin/stdout or UDS is universally implementable; every language can do sockets and JSON.[^9][^1]

- **Extensibility**: using a transport-agnostic canonical envelope allows future addition of NATS, gRPC, or others without breaking clients.[^16][^9]

Bus model

Zorn Mesh should behave as a **hybrid**:

- **Message bus**: subject-based routing, request/reply semantics, load-balanced consumption similar to NATS queue groups.[^23][^17]
- **Event bus**: broadcast topics, fan-out events, and system events (agent-joined, agent-left, etc.).[^5]

Not a traditional **queue system** like RabbitMQ (no complex routing topologies, no global transaction semantics) for MVP. Instead, implement simple per-subscription queues and dead-letter channels inside the daemon.[^14]

***

3. Agent interoperability

Aligning with MCP, A2A, ACP

- **MCP (Model Context Protocol)**: defines how LLM hosts connect to tool servers using JSON-RPC and a standard capability schema.[^24][^25][^26][^1]
- **A2A**: focuses on agent-to-agent communication with a canonical data model (Task, Message, AgentCard, Artifact) and multiple protocol bindings (JSON-RPC, gRPC, HTTP).[^27][^21][^9]
- **ACP**: aims to be the HTTP for AI agents, using REST/HTTP as the transport and standardized JSON message structures.[^28][^10][^16]

For Zorn Mesh, which is local-first and transport-agnostic, the best strategy is:

- **Adopt JSON-RPC 2.0** as the base wire protocol, aligning with MCP and A2A.
- **Define a Zorn canonical envelope and agent model** that can losslessly map to MCP capabilities, A2A AgentCards, and ACP messages.
- Provide **bridges/adapters**:
  - MCP adapter: treat Zorn as a host that can connect to MCP servers or expose MCP-compatible capabilities.[^24][^1]
  - A2A adapter: map Zorn envelopes to A2A Messages and Tasks for cross-system interoperability.[^21][^9]
  - ACP adapter: map to/from ACP HTTP calls for agents that speak ACP only.[^10][^28]

Compatibility vs. Zorn-specific protocol

- **Compatible portions**:
  - JSON-RPC framing and error codes.
  - Core fields like `id`, `method`, `params`, `result`, and standard error structures.[^1][^9]
  - Capability description fields that align with MCP (`name`, `description`, `input_schema`, `output_schema`).[^24][^1]
- **Zorn-specific portions**:
  - Canonical envelope fields for local routing (agent IDs, channels, stream IDs, correlation IDs, trace IDs).
  - Reliability metadata (delivery attempt counts, idempotency keys, ack/nack semantics).
  - Security metadata (agent token, signatures, trust level, process identity claims).

Agent capability model and handshake

Each agent should implement a simple lifecycle with Zorn Mesh:

1. **Connect** via UDS or HTTP/WebSocket.
2. **Register** by sending `agent.register` with:
   - `agent_id` (string, unique within host).
   - `display_name`.
   - `version`.
   - `runtime` (e.g., `python3.11`, `node20`).
   - `capabilities`: list of capability descriptors:
     - `name`: string key.
     - `kind`: `tool` | `service` | `event-consumer` | `event-producer`.
     - `schema`: JSON Schema/TypeBox for input/output.
     - `patterns`: supported messaging patterns: `request`, `event`, `stream`.
     - `constraints`: rate limits, max concurrency.
   - `mcp_compatible`: optional MCP metadata, if the agent is also an MCP server/client.[^24][^1]
3. Zorn Mesh stores this in the **agent registry** and returns a `register.ok` with assigned internal ID and initial leases.
4. Agent sends **heartbeats** (`agent.heartbeat`) periodically to indicate liveness, including current load metrics.
5. Zorn Mesh exposes discovery APIs:
   - `agent.list` (filters by capability, tags).
   - `agent.describe(agent_id)` returns capability descriptors.

Message contract (high level)

- Agents never address each other by raw socket; they address logical **agents** and **capabilities** via subjects like `agent://<agent_id>/apability>`.
- Zorn Mesh is responsible for routing and delivery.

***

4. Protocol and schema design

Canonical message envelope

A suggested envelope (JSON) for all application messages:

```json
{
  "envelope_version": "1.0.0",
  "message_id": "uuid-v4",
  "parent_id": "uuid-v4 or null",
  "correlation_id": "uuid-v4",
  "trace_id": "uuid-v4",
  "span_id": "uuid-v4",
  "timestamp": "RFC3339",
  "from_agent": "agent-id",
  "to": {
    "type": "agent" | "broadcast" | "topic" | "group",
    "agent_id": "optional agent-id",
    "topic": "optional subject string",
    "group": "optional group name"
  },
  "routing": {
    "pattern": "request" | "response" | "event" | "stream" | "ack" | "nack",
    "subject": "string",
    "reply_to": "optional subject or agent capability",
    "ttl_ms": 5000,
    "delivery_mode": "at_most_once" | "at_least_once",
    "attempt": 1
  },
  "identity": {
    "principal": "agent-id",
    "uid": 1000,
    "gid": 1000,
    "pid": 12345,
    "token_id": "optional signed token id",
    "roles": ["dev-tool", "ide", "local-agent"],
    "trust_level": "system" | "trusted" | "untrusted"
  },
  "capability": {
    "name": "code_edit",
    "version": "semver",
    "schema_version": "semver"
  },
```

```
  "reliability": {
    "idempotency_key": "string",
    "dedupe_window_ms": 60000,
    "max_retries": 3,
    "retry_backoff_ms": 1000
  },
  "security": {
    "signature": "base64",
    "key_id": "string",
    "alg": "ed25519",
    "nonce": "optional",
    "anti_replay_window_ms": 600000
  },
  "payload": {
    "type": "json" | "binary" | "text",
    "schema_ref": "urn:zorn:schema:...",
    "data": {}
  },
  "errors": [],
  "meta": {
    "tags": ["build", "refactor"],
    "comments": "optional",
    "user_session_id": "optional"
  }
}
```

This is transport-agnostic; when using JSON-RPC, the `payload` can be embedded in `params` or `result` and the envelope fields included as top-level JSON-RPC properties.

Required fields

- **Identity**: `from_agent`, `identity` block (UID, PID, roles, trust level), and optional signed tokens.[^15][^16]
- **Routing**: `to`, `routing.subject`, `routing.pattern`, `reply_to` for request/response.
- **Correlation/Tracing**: `correlation_id`, `trace_id`, `span_id`, `parent_id` for linking chains.[^20][^2][^19]
- **Topic/versioning**: `capability.name`, `capability.version`, `capability.schema_version`.
- **Reliability**: `reliability` block with idempotency keys, attempts, and retry policy.
- **Security**: a `security` block with signature and key ID for agents that enable signing; optional for MVP.
- **Timestamps**: `timestamp` in RFC3339 for ordering and audit logs.[^11]

Message types

Define type tags in `routing.pattern` or `payload.type`:

- `register`: agent registers itself and capabilities.
- `heartbeat`: liveness message.
- `request`: request from one agent to capability of another.
- `response`: response to a request.
- `event`: fire-and-forget notification.
- `stream_chunk`: chunk in a streaming response.
- `ack`: explicit acknowledgement.
- `nack`: negative acknowledgement (with error info).
- `error`: explicit error event (system-originated).
- `cancel`: request to cancel a stream or long-running request.
- `shutdown`: agent shutting down gracefully.

These can be encoded as `payload.subtype` or `meta.type` rather than separate top-level message kinds.

Versioning and backwards compatibility

- **Envelope version**: `envelope_version` with semantic versioning (e.g., 1.0.0). Major bumps only for breaking changes.
- **Capability schema version**: per-capability `schema_version` and `capability.version` to manage evolution independently.[^21][^9]
- **Compatibility rules**:
  - MVP: require agents to support their declared schema versions and reject unknown majors with a clear error.
  - Zorn Mesh can perform basic **schema translation** between minor versions if adapters are registered.

Schema tooling

For schema definitions and validation:

- Use **JSON Schema** or a TypeScript-friendly variant (e.g., TypeBox/Zod) for agent-visible payloads and capabilities.[^1][^24]
- For the internal canonical model and cross-language code generation, use **Protocol Buffers** for the envelope and registry structures, generating Rust, TS, Go, and Python types.[^9][^21]

Recommended combination:

- **Protobuf**: canonical internal structures and on-disk representation.
- **JSON**: wire format for UDS/JSON-RPC for readability and interoperability.
- **JSON Schema / TypeBox/Zod**: capability input/output contracts for agents.

***

5. Reliability and fault tolerance

Delivery semantics

For a single-host local-first system, **at-least-once** semantics provide a good balance between reliability and complexity:[^14][^5]

- Messages are persisted to an append-only log before delivery (for critical channels).
- Delivery attempts may be retried; consumers must be idempotent.
- Exactly-once is not pursued in MVP; it is expensive to implement and rarely necessary for coding agents.

For non-critical telemetry events, **at-most-once** semantics can be acceptable to avoid backpressure.

Idempotency strategy

- Require agents to accept an `idempotency_key` and treat duplicate requests within a dedupe window as no-ops or return last result.
- Zorn Mesh tracks `(idempotency_key, to_agent, subject)` in a small in-memory or SQLite cache to drop duplicates.

Retries, acknowledgements, timeouts, and dead letters

- **Ack model**:
  - For request/response: implicit ack via response; if timeout expires, the daemon can retry or surface error.
  - For events: optional explicit ack; if requested, daemon expects an `ack` or `nack` message.
- **Timeouts**:
  - Each request carries a `ttl_ms`; daemon stops delivering and marks as dead if exceeded.
- **Retries**:
  - Configurable per capability: `max_retries`, `backoff`.
  - For `nack` with `retryable: false`, message is sent to dead-letter directly.
- **Dead-letter queues**:
  - Per subject/capability, stored in SQLite in a `dead_letters` table.
  - CLI and dashboard offer inspection and manual replay.

Handling agent failures and resource issues

- **Crashed agents / restarts**:
  - Heartbeats missing → mark agent as `offline` and stop delivering new messages.

  - Pending requests can be failed immediately or re-routed to other agents with the same capability.
- **Partial responses / streaming**:
  - Streams carry a `stream_id`; incomplete streams are cleaned up after inactivity timeout.
  - Client can send `cancel` to stop streams.
- **Stuck consumers / backpressure**:
  - Per-agent outbound queue limits; when exceeded, daemon can:
    - Apply local backoff and eventually drop low-priority messages.
    - Send flow-control signals (`slow-consumer` errors) to senders.
- **Local resource exhaustion**:
  - Bounded log size and per-agent quotas.
  - Soft and hard limits on open connections and in-flight messages.

This yields a **realistic reliability model** for local-first without distributed consensus or complex clustering.

***

6. Persistence and state

Persistence options

Options for local message history:

- **In-memory only**: simplest and fastest but no history or replay and fragile on daemon restart.
- **Append-only log files**: simple and high-throughput; harder to query selectively.
- **SQLite**: local, ACID, file-based DB with good tooling and performance, widely used in local-first systems.[^13][^12][^11]
- **LMDB**: memory-mapped KV store, very fast for reads but more complex for ad-hoc queries and multi-writer patterns.[^18]
- **LiteFS-style**: replicated SQLite for multi-node setups, overkill for single-machine MVP.[^12]

Recommendation: SQLite as the primary persistence engine.

- Single file, transactional, easy backup, good integration with local-first patterns and CRDT sync if you later add remote replication.[^12][^11]

What to persist in MVP

Persist:

- Agent registry and capabilities (agents, capabilities, schemas).
- Messages:
  - Critical channels (requests/responses, important events) in append-only tables.

  - Dead-letter messages and metadata.
- Observability:
  - Message metadata (IDs, timestamps, routing, sizes, errors).
  - Audit logs of registration and security events.

Keep ephemeral/in-memory:

- In-flight queues.
- Short-term dedupe caches.
- Live metrics.

Storage layout

SQLite schema sketch:

- `agents(id, agent_id, display_name, version, runtime, trust_level, last_heartbeat_at, status)`
- `capabilities(id, agent_id, name, kind, schema_version, input_schema_json, output_schema_json)`
- `messages(id, message_id, correlation_id, trace_id, parent_id, from_agent_id, to_agent_id, subject, pattern, timestamp, status, attempts, size_bytes, envelope_json)`
- `dead_letters(id, message_id, reason, failed_at, envelope_json)`
- `leases(id, agent_id, capability_id, acquired_at, expires_at)`
- `audit_log(id, timestamp, actor_agent_id, action, details_json)`

Retention, cleanup, and replay

- **Retention policies** per namespace/subject (e.g., keep 24 hours or N messages).
- Background **compaction job** to purge old rows, optionally moving summaries to `audit_log`.
- **Replay**:
  - CLI: `zorn replay --subject agent://foo/code_edit --since 10m` replays matching messages to a test agent.
  - Dashboard: timeline view with filters over the `messages` table.

***

7. Security model

Local-first security principles

Local projects like system NATS have explored using UDS and OS identity to authenticate clients and act as an identity broker. Agent-specific protocols like ACP and A2A also emphasize zero-trust, authenticated communication even between agents.[^10][^16][^15]

For Zorn Mesh, practical principles:

- Assume **untrusted local processes** may exist (malicious code, compromised developers tools).
- Treat Zorn Mesh as a **privileged boundary** mediating access between agents and powerful tools.
- Favor **default-deny** policies, where new agents get limited privileges until explicitly granted.

Authentication

- Core mechanism: **UDS + OS identity** (UID, GID, PID) obtained from the socket for Linux/macOS.[^8][^15]
- Optional second factor:
  - Per-agent **registration tokens** or API keys stored in config files.
  - Short-lived **capability tokens** minted by Zorn Mesh to delegate specific actions.
- For HTTP gateway: use loopback only, random secret and CSRF token for browser-based dashboard.

Authorization and process trust

- Agents are assigned a **trust_level** and roles at registration (`system`, `trusted`, `untrusted`).
- Policy engine (initially simple config file, later upgradable) decides:
  - Which agents can talk to which capabilities.
  - Which agents may issue high-risk requests (filesystem, shell, network calls).
- Process metadata (PPID chain, path, setuid status) can be consulted where platforms allow, similar to proposals for system NATS identity verification.[^15]

Message tampering, replay, and injection

- **Tampering**:
  - UDS and local TCP do not traverse untrusted networks; main risk is compromised local process.
  - Optional message signatures (`security.signature`) using per-agent keys; daemon verifies on ingress.
- **Replay attacks**:
  - Each message carries a nonce and timestamp; daemon rejects messages older than a configurable window or duplicate nonces.[^16]
- **Prompt injection via agents**:
  - Out of scope to fully solve, but Zorn Mesh can label messages with `trust_level` and `source` metadata so consuming agents can apply mitigations.

Malicious adapters and command execution risk

- Agents that expose high-privilege capabilities (e.g., `shell.exec`) must be explicitly whitelisted.

- Zorn Mesh can track **audit trails** of which agent invoked which capability and on what arguments.
- Optionally require **human approval** for certain capabilities, inspired by MCP tool approval flows.[^26]

MVP vs. later hardening

MVP security:

- UDS with strict filesystem permissions.
- OS identity capture and simple role-based policies.
- Registration tokens for agents.
- Audit logging for all registration and high-privilege actions.

Later hardening:

- Per-agent key pairs and signed envelopes.
- TPM-backed keys and secure bootstrapping.[^15]
- Fine-grained policy language (e.g., Rego/OPA integration).
- Sandboxing helpers for untrusted agents.

***

8. Developer experience

API surface

Recommended surfaces:

- **Local daemon** (`zorn-meshd`) as single entrypoint.
- **SDKs**:
  - TypeScript: Node runtime, first-class SDK for VS Code, web, and desktop tools.
  - Python: for local agents and scripts.
  - Optional Rust/Go client libs.
- **CLI**: `zorn` for daemon control and inspection.
- **Web dashboard**: optional Next.js/React app for live observability.

Onboarding flow for a new agent runtime

1. Install Zorn Mesh (binary or brew-like package) and start daemon: `zorn daemon start`.
2. In the agent runtime:
   - Install SDK (`npm install @zorn/mesh` or `pip install zorn-mesh`).
   - Initialize client with UDS path or HTTP URL.
   - Call SDK `registerAgent({...})` with capabilities.

3. Zorn Mesh returns an assigned ID and capability summary.
4. Agent subscribes to subjects for its capabilities (e.g., `agent://my-agent/code_edit`).
5. Testing:
   - Use CLI: `zorn send --to my-agent --capability code_edit --payload ...`.

Inspection tools

- **CLI tools**:
  - `zorn agents` list agents and status.
  - `zorn capabilities` list capabilities.
  - `zorn tail --subject ...` stream messages.
  - `zorn dlq` inspect dead-letter queues.
  - `zorn traces --correlation <id>` show message chains.
- **Dashboard views**:
  - Live topology: agents and links.
  - Message timeline per agent or correlation ID.
  - Health panel: heartbeats, error rates, queue depths.

Local multi-agent workflow

- Developers run agents as separate processes on their machine, each connecting to `zorn-meshd`.
- Provide a `zorn scenario` config format to define test setups:

```yaml
scenario: refactor-demo
agents:
  - name: ide
    command: "node demo-ide.js"
  - name: copilot
    command: "python copilot_agent.py"
  - name: linter
    command: "python lint_agent.py"
flows:
  - description: "Refactor flow"
    steps:
      - send: {from: ide, to: copilot, capability: propose_refactor, payload: {...}}
      - expect: {from: copilot, to: ide, pattern: response}
```

CLI can orchestrate this, collect logs, and run assertions.

***

9. Technical stack

Core language/runtime comparison

- **TypeScript/Node**:
  - Pros: fast to iterate, rich ecosystem, easy to build dashboard, good JSON tooling.[^6]
  - Cons: higher runtime overhead, GC pauses, harder to get robust low-level IPC and resource management.
- **Go**:
  - Pros: strong concurrency model, simple deployment, good networking libraries.
  - Cons: less ergonomic for FFI with MCP/TypeScript ecosystems and more opinionated runtime.
- **Python**:
  - Pros: popular in ML/AI, but not well suited for a high-throughput daemon.
- **Rust**:
  - Pros: performance, safety, control over memory, strong ecosystem for async IO and embedded databases.[^13][^18]
  - Cons: slower developer iteration, steeper learning curve.

Recommendation: **Rust** for the core daemon.

- Rust is appropriate where reliability, resource control, and concurrency matter, similar to how high-performance brokers (e.g., NATS server in Go, other systems in Rust) are built.[^17][^5]
- Bindings for UDS, JSON, and SQLite are mature.

Runtime choices

- If TypeScript is used for any core components, use **Node.js LTS** over Bun/Deno for stability and ecosystem maturity.[^6]
- Bun and Deno are interesting but less battle-tested in production for long-lived daemons; they can be used for auxiliary tools or experimental SDK variants.

Stack recommendations

- **Core runtime (daemon)**: Rust, using:
  - `tokio` or `async-std` for async IO.
  - `serde`/`serde_json` for JSON.
  - `tonic` + `prost` if gRPC is added later.
- **Transport server**:
  - UDS listener with JSON-RPC implementation.
  - Optional Axum/Warp-based HTTP/WebSocket gateway in Rust.
- **Persistence**:
  - SQLite via `rusqlite` or `sqlx`.
- **CLI**:

  - Rust binary (`clap` or `structopt`) or separate Node CLI that shells out to daemon APIs.
- **Web dashboard**:
  - Next.js/React app talking over WebSocket/HTTP to the daemon.
- **Testing / property-based testing**:
  - Rust: `proptest` / `quickcheck` for protocol invariants.
  - Integration tests orchestrated via `cargo test` plus scenario YAML.
- **Schema validation**:
  - Rust: `schemars` for JSON Schema generation.
  - TypeScript: `zod` or `TypeBox` for validating payloads in SDKs.[^24][^1]
- **Logging and tracing**:
  - Rust: `tracing` + `tracing-subscriber`, with JSON log output.
  - Expose OpenTelemetry-compatible traces if desired.[^19][^20]

SDKs

- **TypeScript SDK** (first-class): Node/Edge support, JSON-RPC client, reconnection, and convenience wrappers.
- **Python SDK**: `asyncio` client for UDS/HTTP.
- Optionally **Go/Rust client** for integration with other tools.

***

10. Repository architecture

Monorepo vs. polyrepo

For an early open-source project with multi-language SDKs and a dashboard, a **monorepo** simplifies cross-cutting changes and versioning.

Recommended top-level layout:

- `/core/` – Rust daemon and internal libraries.
- `/protocol/` – Protobuf/JSON Schema definitions and docs.
- `/sdk/ts/` – TypeScript SDK.
- `/sdk/python/` – Python SDK.
- `/cli/` – CLI (could be part of `core` or separate wrapper).
- `/dashboard/` – web UI (Next.js).
- `/examples/` – sample agents in TS/Python/Rust.
- `/tests/` – integration and scenario tests.
- `/docs/` – project documentation.

Package boundaries and naming

- `core/zorn-core` – daemon executable and internal modules.

- `core/zorn-protocol` – shared Rust types for envelopes and schemas.
- `sdk/ts/zorn-mesh` – npm package.
- `sdk/python/zorn_mesh` – PyPI package.
- `dashboard/zorn-dashboard` – app package.

Tooling and CI

- **Build**:
  - Rust: `cargo` workspaces.
  - Dashboard: `pnpm` or `yarn` workspaces.
- **Lint/format**:
  - Rust: `rustfmt`, `clippy`.
  - TS: `eslint`, `prettier`.
  - Python: `ruff`, `black`.
- **CI**:
  - GitHub Actions with matrix for Rust, Node, and Python.
- **Release and versioning**:
  - Semantic versioning per artifact.
  - Tag releases and publish binaries via GitHub Releases.

Example organization

- `/examples/ide-copilot` – VS Code extension + agent that uses Zorn Mesh.
- `/examples/cli-linter` – CLI tool that acts as agent.
- `/examples/mcp-bridge` – bridging MCP servers into Zorn Mesh.[^25][^1][^24]

***

11. Testing strategy

Types of tests

- **Unit tests**:
  - Envelope parsing, validation, routing decisions.
  - SQLite persistence and query correctness.
- **Integration tests**:
  - Multiple simulated agents connecting and exchanging messages.
  - Failure scenarios (agent crash, slow consumer, dead-letter routing).
- **Property-based tests**:
  - Using `proptest` to generate random message graphs and validate invariants (no lost messages beyond retention, correlation ID consistency).
- **Chaos and failure injection**:
  - Randomly kill agent processes and check that daemon cleans up leases and outstanding requests.

  - Inject latency and random nacks.
- **Fuzz testing**:
  - Fuzz JSON envelope parsing and protocol handling to harden against malformed inputs.
- **Load tests**:
  - Local stress tests with many agents and high message throughput to validate backpressure behavior.

Simulating multiple agents locally

- Use a test harness that spawns many child processes or async tasks, each with its own agent ID.
- Provide a `zorn test` command that reads a scenario YAML and orchestrates agents.

Contract testing

- Define a **protocol compliance test suite**:
  - Agents must respond correctly to registration, heartbeats, request/response, and error handling flows.
  - Provide a reference agent that runs tests against adapters.

MVP reliability criteria

Before declaring MVP reliable:

- No message loss in tests beyond configured retention.
- Correct behavior under agent crash and restart (no daemon crash, clean state transitions).
- Consistent delivery semantics (at-least-once) confirmed via property tests.
- Reasonable performance (latency and throughput) for typical coding workloads.

***

12. Observability

Logging, metrics, and traces

Essential observability signals for local-first messaging:

- **Structured logs**:
  - Every message ingress/egress with `message_id`, `from_agent`, `to`, `subject`, `pattern`, `correlation_id`, `status`.
  - Security and policy decisions (auth failures, denied actions).
- **Metrics**:
  - Per-agent: messages sent/received, error rate, average latency, queue depth.
  - Global: total throughput, dead-letter rates, dropped messages, CPU/memory usage.

- **Tracing**:
  - Use `trace_id`/`span_id` fields to build timelines of flows across agents.
  - Optionally integrate with OpenTelemetry to export traces to local collectors.[^20][^19]

CLI and dashboard exposure

- CLI:
  - `zorn logs --agent foo` show recent structured logs.
  - `zorn metrics` summarize key metrics.
  - `zorn trace <trace_id>` display textual waterfall.
- Dashboard:
  - Message timeline per correlation ID with color-coded agent spans.
  - Health/status page for agents.
  - Filters for error types and subjects.

***

13. MVP design and roadmap

Smallest credible architecture

The minimal but credible MVP architecture:

```mermaid
flowchart LR
    subgraph Host[Developer Machine]
        ZD[zorn-meshd (Rust)]
        DB[(SQLite)]
        CLI[Zorn CLI]
        UI[Web Dashboard]
        A1[Agent 1
 (IDE plugin)]
        A2[Agent 2
 (Copilot bridge)]
        A3[Agent 3
 (Local tool)]
    end

    A1 -- UDS JSON-RPC --> ZD
    A2 -- UDS JSON-RPC --> ZD
    A3 -- HTTP/WebSocket --> ZD
    ZD <---> DB
    CLI -- HTTP/UDS --> ZD
    UI -- WebSocket/HTTP --> ZD
```

```

### MVP feature set

- Core daemon with UDS JSON-RPC API.
- SQLite-backed agent registry and message metadata.
- Request/response and pub/sub patterns.
- Basic at-least-once delivery with dead letters.
- UDS-based security and simple RBAC.
- TypeScript and Python SDKs.
- CLI with basic inspection and replay.

**Non-goals for MVP**:

- Multi-host clustering.
- Advanced zero-trust key management.
- Complex workflows and planners.
- Deep CRDT-powered shared state.

### Roadmap

- **MVP (v0.1)**:
  - Single host, UDS/HTTP transports, basic messaging and persistence, CLI, TS/Python SDKs.
- **v0.2**:
  - Web dashboard, better observability, dead-letter replay tooling.
  - MCP/A2A bridge adapters.[^9][^1]
- **v0.5**:
  - Pluggable transports (optional NATS integration, gRPC option).
  - Stronger security (per-agent keys, signatures).
  - Scenario-based testing framework.
- **v1.0**:
  - Stable protocol and schemas.
  - Hardened security and reliability guarantees.
  - Rich ecosystem of example agents and IDE integrations.

### Biggest design mistakes to avoid

- Overcommitting to a single external standard (e.g., MCP) such that Zorn becomes just a thin wrapper instead of a general local mesh.[^3][^1]
- Overengineering distribution (multi-node, consensus) before nailing local reliability and ergonomics.[^5]
- Making transport choices hard to change (binding application semantics directly to UDS paths or HTTP URIs instead of a canonical envelope).[^21][^9]
```

- Neglecting observability and introspection; debugging agent flows must be a first-class experience.[^2][^19][^20]

***

14. Final recommendation

Recommended architecture (summary)

- A **Rust-based local daemon** (`zorn-meshd`) exposing a **UDS JSON-RPC 2.0 API** for agents and an optional HTTP/WebSocket gateway for dashboards and non-UDS runtimes.[^7][^6][^1]
- A **canonical message envelope** stored in **SQLite**, with an agent registry and message metadata tables for history and replay.[^13][^11][^12]
- A **hybrid message/event bus core**, with at-least-once delivery for critical messages and at-most-once for telemetry.[^14][^5]
- A **local-first security layer** built on UDS permissions, OS identity, and optional per-agent tokens and signatures.[^16][^15]

Recommended technical stack

- Core: Rust (`tokio`, `serde`, `rusqlite`/`sqlx`, `tracing`).[^18][^13]
- Protocol: Protobuf for canonical types; JSON/JSON-RPC over UDS/HTTP for wire.
- Persistence: SQLite.
- CLI: Rust (same binary) with `clap`.
- Dashboard: Next.js/React, talking WebSocket/HTTP to the daemon.
- SDKs: TypeScript (Node) and Python (asyncio), with JSON-RPC clients.

Recommended repository structure

- Monorepo with `/core`, `/protocol`, `/sdk/ts`, `/sdk/python`, `/dashboard`, `/examples`, `/tests`, `/docs`.

Recommended reliability model

- At-least-once delivery for application messages, relying on idempotency keys and retries.
- Dead-letter queues and replay for observability and debugging.
- Bounded queues and backpressure mechanisms.

Recommended security model

- Local-only UDS communication by default, with strict socket permissions.
- OS identity-based authentication and role assignment.
- Optional signed envelopes and per-agent keys for sensitive capabilities.

- Audit logging and capability-level allowlists.

***

Decision matrix (major alternatives)

```markdown
| Dimension          | Option                      | Pros                                                | Cons
| Recommendation |
|-------------------|-----------------------------|-----------------------------------------------------|---------------
-----------------------------------------|----------------|
| Core language      | Rust                        | Performance, safety, strong async & DB ecosystem
| Steeper learning curve                              | ✅ Use        |
|                    | Go                          | Simple, fast, great net libs                        | Less control,
different ecosystem than agent tooling     | Consider later |
|                    | TypeScript/Node             | Fast dev, rich ecosystem                            | Higher
overhead, long-lived daemon stability concerns    | No            |
| Primary transport  | UDS + JSON-RPC              | Low latency, secure, MCP/A2A aligned
| Non-UDS envs need gateway                           | ✅ Use        |
|                    | TCP/HTTP only               | Universal, easy debugging                           |
Higher overhead, weaker local security                | Gateway only   |
| Messaging backend  | Custom bus over SQLite      | Tailored semantics, simple local-first
persistence       | Needs implementation effort                         | ✅ Use        |
|                    | NATS                        | Mature broker, rich features                        | Extra
process, config, overkill for single host          | Optional later |
| Persistence        | SQLite                      | ACID, local-first, familiar, easy querying          | Not
optimized for extreme write throughput             | ✅ Use        |
|                    | LMDB                        | Ultra-fast reads                                    | Harder ad-hoc
queries, write constraints                | Maybe later    |
| Interop standard   | JSON-RPC + MCP/A2A/ACP bridge | Aligns with ecosystem, flexible,
canonical envelope      | Requires adapters                                   | ✅ Use        |
| Security           | OS identity + tokens        | Practical local model, simple to operate            |
Weaker than full zero-trust with HW keys              | ✅ MVP        |
|                    | Per-agent key pairs + TPM   | Strong identity, zero-trust                         |
Complex bootstrapping & UX                            | v0.5+         |
```

***

MVP bill of materials

- `zorn-meshd` Rust daemon:
  - UDS JSON-RPC server.
  - SQLite persistence (agents, messages, dead letters, audit).

  - Core routing and reliability logic.
  - UDS security and basic RBAC.
- TypeScript SDK:
  - JSON-RPC client over UDS/HTTP.
  - Agent registration helpers.
  - Request/response, pub/sub, and streaming APIs.
- Python SDK:
  - Asyncio-based client.
- CLI (`zorn`):
  - Start/stop daemon, list agents, tail messages, inspect dead letters, replay.
- Minimal Web UI:
  - Agent list and message timeline.
- Protocol definitions:
  - Protobuf and JSON Schema for envelope and capabilities.
- Example agents:
  - IDE plugin, simple local tool, MCP bridge.

***

"Build this now" blueprint

1. **Define protocol and schemas**:
   - Finalize the canonical envelope and agent registration messages in Protobuf and JSON Schema.
   - Implement JSON-RPC bindings for register, heartbeat, send, subscribe, and admin APIs.
2. **Implement core daemon (Rust)**:
   - UDS listener with JSON-RPC handling.
   - SQLite-backed agent registry and message metadata.
   - In-memory routing tables with per-agent queues.
   - At-least-once delivery with retries, dead letters, and basic backpressure.
   - Structured logging via `tracing` and metrics via internal counters.
3. **Add security and policy**:
   - Enforce UDS permissions and capture UID/GID/PID.
   - Implement token-based authentication and simple role-based policies.
   - Log all security decisions to audit log.
4. **Build SDKs (TS & Python)**:
   - Provide a simple `registerAgent()` API and message handlers.
   - Support request/response, events, and streaming with correlation IDs.
5. **Create CLI and minimal dashboard**:
   - CLI for control and inspection.
   - Web UI (Next.js) for topology view and message timelines.
6. **Ship example agents**:
   - A demo IDE-like agent that sends code context and receives refactor suggestions.

- A bridge agent that connects to an MCP server and exposes it inside Zorn Mesh.[^25][^26][^1]
7. **Harden via tests**:
   - Unit, integration, property, chaos, and fuzz tests to validate reliability invariants and security.

With this blueprint, an engineer can begin implementing Zorn Mesh as a serious, evolvable local agent communication platform that aligns with emerging standards while staying focused on the single-host, developer-centric use case.

References

1. [Specification - Model Context Protocol](https://modelcontextprotocol.io/specification/2025-06-18) - Model Context Protocol (MCP) is an open protocol that enables seamless integration between LLM appli...

2. [Dapr and service meshes](https://docs.dapr.io/concepts/faq/service-mesh/) - While Dapr and service meshes do offer some overlapping capabilities, Dapr is not a service mesh, wh...

3. [Model Context Protocol - GitHub](https://github.com/modelcontextprotocol) - The Model Context Protocol (MCP) is an open protocol that enables seamless integration between LLM a...

4. [Dapr vs Linkerd: Application Runtime vs Service Mesh - OneUptime](https://oneuptime.com/blog/post/2026-03-31-dapr-vs-linkerd-service-mesh/view) - Compare Dapr and Linkerd to understand their complementary roles - Dapr as an application runtime an...

5. [redis - Brave New Geek](https://bravenewgeek.com/tag/redis/)

6. [The Node.js Developer's Guide to Unix Domain Sockets - Node Vibe](https://nodevibe.substack.com/p/the-nodejs-developers-guide-to-unix) - Unix domain sockets deliver ~50% lower latency than TCP loopback for Node.js IPC. We got the 334µs l...

7. [communication between processes: tcp vs unix sockets, ipc vs nats](https://stackoverflow.com/questions/41447064/communication-between-processes-tcp-vs-unix-sockets-ipc-vs-nats/41495084) - I'm breaking a big application into several processes and I want each process to communicate with ea...

8. [Unix Domain Sockets vs Loopback TCP Sockets (2014) | Hacker News](https://news.ycombinator.com/item?id=37466475) - Always choose unix domain sockets over local TCP if it is an option. There are some valid reasons th...

9. [Agent2Agent (A2A) Protocol Specification](https://a2a-protocol.org/latest/specification/) - The official documentation for the Agent2Agent (A2A) protocol. The A2A protocol is an open standard ...

10. [ACP Explained: The Vendor-Neutral Backbone for Cooperative AI](https://www.educative.io/newsletter/artificial-intelligence/agent-communication-protocol) - The Agent Communication Protocol (ACP) gives autonomous agents a shared, vendor-neutral “language” s...

11. [The Secret Life of a Local-First Value - Marco Bambini](https://marcobambini.substack.com/p/the-secret-life-of-a-local-first) - A deep dive into how CRDT-powered local-first apps track, merge, and sync every INSERT, UPDATE, and ...

12. [Local First SQLite - Turso](https://turso.tech/local-first) - Turso is a great choice to build local first apps. It's based on SQLite so your database is just a f...

13. [Database Comparison: DuckDB, SQLite, and LevelDB - GitHub](https://github.com/marvelousmlops/database_comparison) - This project compares 3 popular databases: DuckDB, SQLite, and LevelDB. The purpose is to compare th...

14. [RabbitMQ vs Redis OSS - Difference Between Pub/Sub ... - AWS](https://aws.amazon.com/compare/the-difference-between-rabbitmq-and-redis/) - RabbitMQ is a message broker, while Remote Dictionary Server (Redis OSS) is an in-memory key-value d...

15. [[UNIX sockets] Extending NATS to become a backbone ... - GitHub](https://github.com/nats-io/nats-server/discussions/7677) - The nats would serve various components to do IPC, communicate state, etc. The reason why this shoul...

16. [Beyond Context Sharing: A Unified Agent Communication Protocol ...](https://arxiv.org/html/2602.15055v1)

17. [Queue Groups | NATS Docs](https://docs.nats.io/nats-concepts/core-nats/queue) - NATS provides an additional feature named "queue", which allows subscribers to register themselves a...

18. [Top 8 Embedded SQL Databases in 2025 - Explo](https://www.explo.co/blog/embedded-sql-databases) - LMDB is a key-value store known for its ultra-fast read performance and minimal memory usage. It use...

19. [Decoding the Dynamics: Dapr vs. Service Meshes](https://techcommunity.microsoft.com/blog/azuredevcommunityblog/decoding-the-dynamics-dapr-vs-service-meshes/4021563) - The purpose of this post is to highlight the differences, especially on the way they handle mTLS, as...

20. [Sidecars or Sharing: A Practical Guide to Selecting Your Service Mesh](https://www.buoyant.io/blog/sidecars-or-sharing-a-practical-guide-to-selecting-your-service-mesh) - Most current meshes, including Linkerd, use sidecar proxies as a way to simplify the operational mod...

21. [Agent2Agent (A2A) Protocol Specification (DRAFT v1.0)](https://a2a-protocol.org/dev/specification/) - The official documentation for the Agent2Agent (A2A) protocol. The A2A protocol is an open standard ...

22. [133. Why Unix Domain Sockets Are Faster Than TCP - YouTube](https://www.youtube.com/watch?v=zYzme6Ollj4) - Are you still using localhost and TCP ports for local communication in your Node.js apps? Discover a...

23. [How to Implement Queue Groups in NATS - OneUptime](https://oneuptime.com/blog/post/2026-02-02-nats-queue-groups/view) - Queue groups allow you to distribute messages among multiple subscribers, ensuring each message is p...

24. [04-model-context-protocol.md - GitHub](https://github.com/microsoft/Generative-AI-for-beginners-dotnet/blob/main/04-AgentsWithMAF/04-model-context-protocol.md) - Model Context Protocol (MCP) is an open standard that defines how applications provide tools and con...

25. [About Model Context Protocol (MCP) - GitHub Docs](https://docs.github.com/en/copilot/concepts/context/mcp) - Model Context Protocol (MCP) is a protocol that allows you to extend the capabilities of GitHub Copi...

26. [Model context protocol (MCP) - OpenAI Agents SDK](https://openai.github.io/openai-agents-python/mcp/) - MCP is an open protocol that standardizes how applications provide context to LLMs. Think of MCP lik...

27. [Announcing the Agent2Agent Protocol (A2A)](https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/) - Explore A2A, Google's new open protocol empowering developers to build interoperable AI solutions.

28. [What is ACP (Agent Communication Protocol)? - Microblink](https://microblink.com/resources/glossary/acp-agent-communication/) - Learn what is ACP agent communication protocol and how it enables AI agents to work together using s...